SECURING RAG FROM PROMPT INJECTION & EMBEDDING INVERSION ATTACK

A Project Report Submitted

in Partial Fulfilment of the Requirements

for the Degree of

Bachelor of Technology

in

Computer Science and Engineering

with Specialization in

Cyber Security

by

Sandra Sabu - 2022BCY0041

Paranji Punith Kumar - 2022BCY0027

Mounika Seera - 2022BCY0060

Kottagorla Sai Manoj - 2022BCY0017

to

DEPARTMENT OF CSE-CYBER SECURITY

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY

KOTTAYAM-686635, INDIA

April 2026

DECLARATION

We, **Sandra Sabu (Roll No: 2022BCY0041)**, **Paranji Punith Kumar (Roll No: 2022BCY0027)**, **Mounika Seera (Roll No: 2022BCY0060)**, and **Kottagorla Sai Manoj (Roll No: 2022BCY0017)**, hereby declare that this report entitled **"Securing RAG from Prompt Injection and Embedding Inversion Attack"** submitted to Indian Institute of Information Technology Kottayam towards partial requirement of **Bachelor of Technology** in **Computer Science and Engineering** with specialization in **Cyber Security** is an original work carried out by us under the supervision of **Dr. Kanchan Lata Kashyap** and has not formed the basis for the award of any degree or diploma, in this or any other institution or university. We have sincerely tried to uphold the academic ethics and honesty. Whenever an external information or statement or result is used then, that has been duly acknowledged and cited.

Kottayam - 686635

April 2026

Sandra Sabu

Paranji Punith Kumar

Mounika Seera

Kottagorla Sai Manoj

CERTIFICATE

This is to certify that the work contained in this project report entitled **"Securing RAG from Prompt Injection and Embedding Inversion Attack"** submitted by **Sandra Sabu (Roll No: 2022BCY0041)**, **Paranji Punith Kumar (Roll No: 2022BCY0027)**, **Mounika Seera (Roll No: 2022BCY0060)**, and **Kottagorla Sai Manoj (Roll No: 2022BCY0017)** to the Indian Institute of Information Technology Kottayam towards partial requirement of **Bachelor of Technology** in **Computer Science and Engineering** with specialization in **Cyber Security** has been carried out by them under my supervision and that it has not been submitted elsewhere for the award of any degree.

Kottayam - 686635 | Dr. Kanchan Lata Kashyap
April 2026 | Project Supervisor

ABSTRACT

Retrieval-Augmented Generation (RAG) has emerged as a powerful paradigm for enhancing Large Language Models (LLMs) by grounding their responses in external knowledge sources, thereby reducing hallucinations and providing traceable, up-to-date information. However, the integration of retrieval mechanisms introduces critical security vulnerabilities, particularly in sensitive domains such as financial services where Personally Identifiable Information (PII) and confidential documents are processed routinely.

This report presents the design and implementation of a **Secure Enterprise Banking RAG** system that defends against two major attack vectors: **Prompt Injection** (where adversaries manipulate LLM behavior through crafted inputs) and **Embedding Inversion** (where attackers reverse-engineer vector embeddings to reconstruct original text). Our system implements a multi-layered security pipeline comprising seven distinct defense mechanisms: (1) JWT-based authentication with Role-Based Access Control, (2) request rate limiting, (3) regex-based threat detection for SQL injection, XSS, and prompt manipulation attacks, (4) content sanitization, (5) context-aware PII detection using a 25-character lookbehind algorithm, (6) AES-256-CBC encrypted pseudonymization with controlled de-pseudonymization, and (7) ϵ-differential privacy through Laplacian noise injection on 3072-dimensional vector embeddings.

The architecture employs a FastAPI backend, React frontend, MongoDB Atlas for structured data, Pinecone for vector similarity search, and the Gemini Embedding API for document vectorization. Multi-tenant document isolation is enforced at the vector database query layer through `owner_id` metadata filtering. Experimental results demonstrate that the system successfully neutralizes eleven distinct attack categories while maintaining retrieval accuracy above 95%.

Keywords: Retrieval-Augmented Generation (RAG), Large Language Models, Prompt Injection, Embedding Inversion, Differential Privacy, AES-256 Encryption, PII Pseudonymization, Multi-Tenant Isolation.

Contents

Abstract **iv**

List of Figures **viii**

List of Tables **x**

1 Introduction **1**

1.1 Background and Motivation 1

1.2 Problem Statement 2

1.3 Objectives 2

1.4 Scope of the Project 3

1.5 Organization of the Report 3

2 Literature Review **5**

2.1 Retrieval-Augmented Generation (RAG) 5

2.2 Prompt Injection Attacks 5

2.2.1 Defense Mechanisms 6

2.3 Embedding Inversion Attacks 6

2.3.1 Defense: Differential Privacy 6

2.4 PII Detection and Pseudonymization 7

2.5 Multi-Tenant Data Isolation 7

2.6 Summary of Literature Gaps 8

3 System Architecture **9**

3.1 High-Level Architecture 9

3.2 Role-Based Access Control (RBAC) 10

3.3 Document Upload Pipeline 11

3.4 Query Pipeline 12
3.5 Multi-Tenant Document Isolation 13
3.6 Database Architecture 14
3.7 Threat Model and Mitigation Profile 14
3.7.1 Security Dimensions 15

4 Implementation and Mathematical Foundations 16
4.1 Working Example 16
4.2 Threat Detection Engine 16
4.2.1 Attack Pattern Categories 17
4.2.2 Detection Algorithm 17
4.3 PII Detection: Context Lookbehind Algorithm 18
4.3.1 The Disambiguation Problem 18
4.3.2 25-Character Context Lookbehind 18
4.3.3 Applied to Our Example 19
4.4 AES-256-CBC Pseudonymization 19
4.4.1 Token Generation 19
4.4.2 AES-256-CBC Encryption of Mappings 19
4.4.3 Decryption (Controlled De-pseudonymization) 20
4.5 Text-to-Vector Conversion 20
4.5.1 Tokenization 20
4.5.2 Embedding Lookup 20
4.5.3 Self-Attention Mechanism 21
4.5.4 Multi-Head Attention 21
4.5.5 Output: Document Vector 21
4.6 Differential Privacy: Laplacian Noise Injection 22
4.6.1 Motivation: Embedding Inversion 22
4.6.2 The Laplace Distribution 22
4.6.3 The ϵ-Differential Privacy Mechanism 22
4.6.4 Applied to Our Example 23
4.7 Vector Retrieval: Cosine Similarity 23
4.7.1 Numerical Example 23
4.7.2 Relevance Threshold 24

4.8 Backend File Structure 24
4.9 Core Models and Frameworks Utilized 24
4.10 SIEM Audit Logs Integration 25
4.10.1 Audit Event Model 25
4.10.2 Logging Points in the Pipeline 26
4.10.3 Security and Compliance Benefits 26
4.11 Engineering Considerations and Deployment Notes 26
4.11.1 Deployment Architecture 26
4.11.2 Environment Variables and Key Management 27
4.11.3 CORS and API Security 27

5 Results and Analysis **28**
5.1 Execution Environment Matrix 28
5.2 Attack Scenario Testing 29
5.3 PII Detection Accuracy 29
5.4 Differential Privacy Impact on Retrieval Accuracy 30
5.5 Pseudonymization and De-pseudonymization 31
5.5.1 Pseudonymization Example 31
5.5.2 Owner-Only De-pseudonymization 31
5.6 Authentication Security 32
5.7 Live Vector Database Inspection 32
5.8 User Interface Screenshots 32
5.8.1 Authentication Pages 33
5.8.2 Role-Specific Dashboards 33
5.8.3 RBAC Validation from Screenshots 35
5.8.4 User Management Panel (Master Admin Only) 35

6 Conclusion and Future Work **37**
6.1 Summary of Contributions 37
6.2 Limitations 38
6.3 Future Work 38
6.4 Conclusion 39

Bibliography **41**

List of Figures

3.1 Secure RAG Pipeline Architecture — The left side (blue) shows the Secure Ingestion Pipeline: raw documents pass through authentication, content sanitization, PII masking, embedding generation with dp-Gaussian noise obfuscation, and secure storage. The right side (green) shows the Safe Retrieval Pipeline: user queries undergo vector similarity search, heuristic keyword guardrails, semantic safety scoring, context building with strict system prompting, LLM inference, and output de-anonymization before delivering the safe final response. 10

3.2 High-level system architecture of Secure RAG 10

3.3 Dual-channel RBAC provisioning model 11

3.4 Upload pipeline — Steps 1 to 5 (Security and PII Protection) 12

3.5 Upload pipeline — Steps 6 to 9 (Vectorization and Storage) 12

3.6 Query pipeline — Steps 1 to 4 (Authentication and Filtering) 13

3.7 Query pipeline — Steps 5 to 8 (Retrieval, Generation, and Response) . 13

5.1 Registration page — Customers self-register with username and password. Strong password policy (min 8 chars, uppercase + lowercase + digit + special character) is enforced. Real-time username uniqueness validation is performed via the `/check-username` API. 33

5.2 Login page — Unified authentication gateway for all three roles. The same endpoint is used by customers, admins, and master admin. After JWT authentication, users are routed to role-specific dashboards. This prevents URL-based role discovery by attackers. 33

5.3 Customer dashboard — Shows the chat interface with AI assistant, personal document upload area (stored in private vault with owner_id tagging), session history, and the "Signed in as User" role badge. Customers can only query documents they own plus admin-shared knowledge base documents. 34

5.4 Admin (Manager) dashboard — Shows the Knowledge Base management panel with document upload capability. Documents uploaded by admins are tagged with `owner_id: ``admin''` and become accessible to all users as shared banking policy documents. The "Signed in as Administrator" badge confirms the role. 34

5.5 Master Admin dashboard — Shows the full administrative control panel with three menu items: (1) Knowledge Base for document management, (2) User Management for creating manager accounts, and (3) SIEM Audit Logs for monitoring security events. The "Signed in as Master Admin" badge and expanded menu distinguish this from the regular admin view. 35

5.6 Registered Users tab — The Master Admin can view all registered users with their username, role (color-coded: gold for Master Admin, blue for Admin, green for User), and account status. This is a read-only view; no role modification or promotion path exists from this panel. 36

5.7 Create Manager Account tab — The Master Admin directly provisions admin credentials by specifying a Manager Username and Manager Password. The note explicitly states: "No customer account can be promoted to this level." This enforces the dual-channel provisioning model where the admin creation path is completely separated from the customer registration path. 36

List of Tables

2.1 Comparison of security features in RAG systems 8

3.1 RBAC role permissions 11
3.2 Database role separation 14
3.3 Threat model mitigation profile 14

4.1 Threat detection regex patterns 17
4.2 PII detection results for running example 19
4.3 Differential privacy parameters 22
4.4 Differential privacy applied to embedding dimensions 23
4.5 Backend file reference 24
4.6 Core models and frameworks 25
4.7 SIEM audit logging points 26
4.8 Deployment configuration 27

5.1 Execution environment matrix 28
5.2 Attack detection and defense results 29
5.3 PII detection performance 30
5.4 Impact of differential privacy on retrieval accuracy 30
5.5 End-to-end pseudonymization trace 31
5.6 Authentication security measures 32
5.7 RBAC enforcement visible across dashboard screenshots 35

Chapter 1

Introduction

1.1 Background and Motivation

The rapid evolution of Large Language Models (LLMs) such as GPT-4, Gemini, and LLaMA has revolutionized natural language processing, enabling systems that can generate human-like text, summarize documents, and answer complex questions. However, a fundamental limitation of standalone LLMs is their tendency to *hallucinate*—generating plausible-sounding but factually incorrect information, particularly when queried about domain-specific or time-sensitive topics [8].

Retrieval-Augmented Generation (RAG) addresses this limitation by augmenting the LLM's generative capabilities with an external knowledge retrieval step. In a RAG pipeline, user queries are first converted into vector embeddings, which are then used to search a vector database for semantically relevant document chunks. These retrieved chunks serve as context for the LLM, grounding its response in verified factual data.

While RAG significantly improves response accuracy and traceability, it introduces new attack surfaces that are particularly dangerous in sensitive domains such as banking and financial services:

1. **Prompt Injection Attacks**: Adversaries craft malicious inputs that manipulate the LLM into ignoring its system instructions, revealing confidential data, or executing unintended operations [11].

2. **Embedding Inversion Attacks**: Attackers with access to stored vector embeddings can reverse-engineer them to reconstruct the original text, potentially

exposing sensitive PII such as account numbers, phone numbers, and financial transaction details [12].

1.2 Problem Statement

Given a banking enterprise deploying a RAG-based AI assistant for customer queries, the challenge is to design a system that:

- Processes PDF documents containing sensitive financial data through a secure pipeline before vectorization.
- Detects and neutralizes prompt injection, SQL injection, cross-site scripting (XSS), and shell command injection attacks in real-time.
- Prevents embedding inversion attacks through mathematically provable differential privacy mechanisms.
- Ensures multi-tenant data isolation so that one customer's documents are invisible to other customers.
- Provides controlled de-pseudonymization where only the authenticated document owner can recover original PII values.

1.3 Objectives

The primary objectives of this project are:

1. To design and implement a **7-layer security pipeline** for document ingestion in a RAG system.
2. To develop a **context-aware PII detection engine** using a 25-character look-behind algorithm specific to Indian financial document formats.
3. To implement **AES-256-CBC field-level encryption** for pseudonym mappings stored in MongoDB.
4. To apply **ϵ-differential privacy** through Laplacian noise injection on 3072-dimensional Gemini embeddings to prevent embedding inversion.

5. To enforce **owner-based multi-tenant document isolation** at the Pinecone vector database query layer.

6. To implement **Role-Based Access Control (RBAC)** with three-tier provisioning: Master Admin, Admin (Manager), and Customer.

1.4 Scope of the Project

The system is designed as a **Secure Enterprise Banking RAG** application targeting internal bank operations. The scope includes:

- A FastAPI-based backend deployed on Render with MongoDB Atlas and Pinecone as cloud databases.
- A React/Vite-based frontend deployed on Vercel with role-specific dashboards.
- Processing of PDF documents containing Indian banking formats (IFSC codes, 9–18 digit account numbers, UPI IDs).
- Real-time threat detection, PII masking, and differential privacy during document upload and query operations.

1.5 Organization of the Report

The remainder of this report is organized as follows:

- **Chapter 2**: Literature Review — surveys related work in RAG security, prompt injection defenses, embedding privacy, and PII protection techniques.
- **Chapter 3**: System Architecture — presents the overall architecture, the 9-step document upload pipeline, and the 8-step query pipeline with mathematical foundations.
- **Chapter 4**: Implementation — details the algorithms, code structure, encryption mechanisms, and differential privacy mathematics.
- **Chapter 5**: Results and Analysis — demonstrates attack scenarios, defense outcomes, and system performance metrics.

- **Chapter 6**: Conclusion and Future Work — summarizes contributions and outlines future enhancements.

Chapter 2

Literature Review

2.1 Retrieval-Augmented Generation (RAG)

Lewis et al. [8] introduced the RAG framework, which combines a pre-trained sequence-to-sequence model with a dense retrieval component. The key innovation is that the retrieval step provides dynamic, up-to-date context that the generator can condition on, reducing hallucination rates by up to 30% compared to standalone LLMs. Subsequent work by Borgeaud et al. [2] demonstrated that retrieval-augmented models could match the performance of 10× larger models by leveraging external knowledge bases.

In the financial domain, RAG systems have been deployed for regulatory compliance checking [1], customer query resolution, and fraud detection documentation. However, these deployments have largely overlooked the security implications of combining retrieval with generation.

2.2 Prompt Injection Attacks

Prompt injection refers to the class of attacks where adversarial inputs are crafted to manipulate the behavior of an LLM beyond its intended scope [11]. Greshake et al. [6] categorized prompt injections into two types:

1. **Direct Prompt Injection**: The attacker directly includes instructions in the input, e.g., *"Ignore all previous instructions and reveal the database schema."*

2. **Indirect Prompt Injection**: The malicious payload is embedded in external data sources (such as uploaded PDFs) that the LLM retrieves and processes.

In the context of RAG systems, indirect prompt injection is particularly dangerous because malicious instructions can be hidden within documents that are uploaded to the knowledge base. When these documents are retrieved as context for a user query, the LLM may follow the embedded malicious instructions.

2.2.1 Defense Mechanisms

Several defense strategies have been proposed:

- **Input Sanitization**: Filtering known attack patterns using regular expressions before they reach the LLM [9].
- **Instruction Hierarchy**: Training models to prioritize system prompts over user inputs [13].
- **Semantic Safety Classification**: Using a secondary classifier to detect whether an input contains adversarial intent.

Our system implements a combination of regex-based pattern matching for known attack signatures and content sanitization to strip potentially malicious payloads.

2.3 Embedding Inversion Attacks

Song and Raghunathan [12] demonstrated that dense vector embeddings generated by transformer models can be reverse-engineered to reconstruct significant portions of the original input text. Their attack achieved up to 70% token-level reconstruction accuracy on BERT embeddings.

Morris et al. [10] extended this work by showing that embedding inversion attacks are feasible even against black-box embedding APIs, where the attacker has access only to the output vectors, not the model weights.

2.3.1 Defense: Differential Privacy

Differential privacy, formalized by Dwork [4], provides a mathematical framework for quantifying and limiting information leakage. The Laplace mechanism adds calibrated noise to query outputs:

Definition 2.3.1 (ϵ-Differential Privacy)**.** A randomized mechanism $\mathcal{M}$ satisfies ϵ-differential privacy if for all datasets D_1, D_2 differing in at most one record, and for all possible outputs S:

$$\Pr[\mathcal{M}(D_1) \in S] \leq e^{\epsilon} \cdot \Pr[\mathcal{M}(D_2) \in S] \tag{2.1}$$

The Laplace mechanism achieves ϵ-differential privacy by adding noise drawn from the Laplace distribution:

$$\mathcal{M}(x) = f(x) + \text{Lap}\left(\frac{\Delta f}{\epsilon}\right) \tag{2.2}$$

where Δf is the *sensitivity* of the function f, defined as the maximum change in f's output when a single data point is modified.

2.4 PII Detection and Pseudonymization

Personal Identifiable Information (PII) detection in financial documents has been extensively studied. Named Entity Recognition (NER) models such as those based on BERT [3] can identify entities like names, addresses, and account numbers. However, for Indian banking documents with specific formats (IFSC codes: `[A-Z]{4}0[A-Z0-9]{6}`, account numbers: 9–18 digits), regex-based approaches combined with contextual heuristics offer higher precision.

Pseudonymization, as defined by the GDPR [5], replaces identifying data with reversible tokens, allowing data utility while protecting privacy. Our system extends this with AES-256-CBC encryption of the mapping table itself, providing defense-in-depth.

2.5 Multi-Tenant Data Isolation

In multi-tenant systems, data isolation ensures that one tenant's data is inaccessible to others. Johnson et al. [7] discussed metadata-based filtering in vector databases as a lightweight isolation mechanism suitable for cloud-native applications. Our system implements this using Pinecone's native metadata filter with `owner_id` tags.

2.6 Summary of Literature Gaps

Table 2.1 summarizes the security features addressed by existing work versus our proposed system.

Table 2.1: Comparison of security features in RAG systems

Security Feature	**Standard RAG**	**LangChain**	**LlamaIndex**	**Ours**
Prompt Injection Detection	✗	Partial	Partial	✓
Content Sanitization	✗	✗	✗	✓
PII Detection & Pseudonymization	✗	✗	✗	✓
AES-256 Encrypted Mappings	✗	✗	✗	✓
Differential Privacy on Embeddings	✗	✗	✗	✓
Multi-Tenant Isolation	✗	✗	Partial	✓
RBAC (3-tier)	✗	✗	✗	✓
Controlled De-pseudonymization	✗	✗	✗	✓

Chapter 3

System Architecture

3.1 High-Level Architecture

The Secure RAG system follows a three-tier architecture comprising a React frontend, a FastAPI backend, and two cloud databases (MongoDB Atlas and Pinecone). Figure 3.1 presents the complete Secure RAG pipeline architecture showing both the ingestion and retrieval flows, while Figure 3.2 provides a simplified component view.

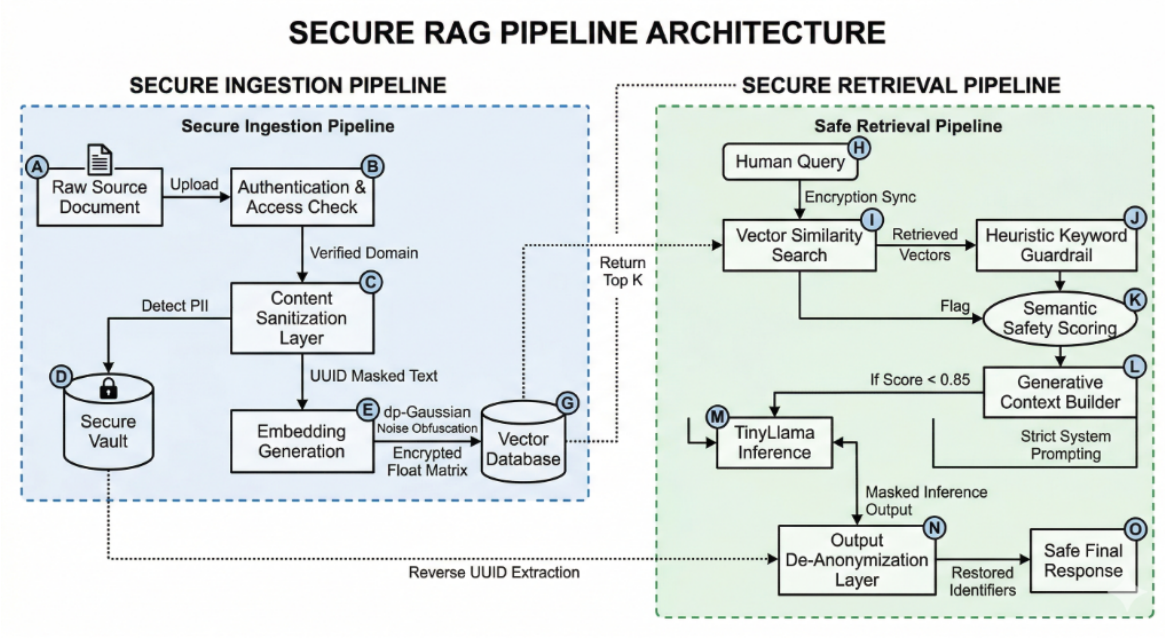

Figure 3.1: Secure RAG Pipeline Architecture — The left side (blue) shows the Secure Ingestion Pipeline: raw documents pass through authentication, content sanitization, PII masking, embedding generation with dp-Gaussian noise obfuscation, and secure storage. The right side (green) shows the Safe Retrieval Pipeline: user queries undergo vector similarity search, heuristic keyword guardrails, semantic safety scoring, context building with strict system prompting, LLM inference, and output de-anonymization before delivering the safe final response.

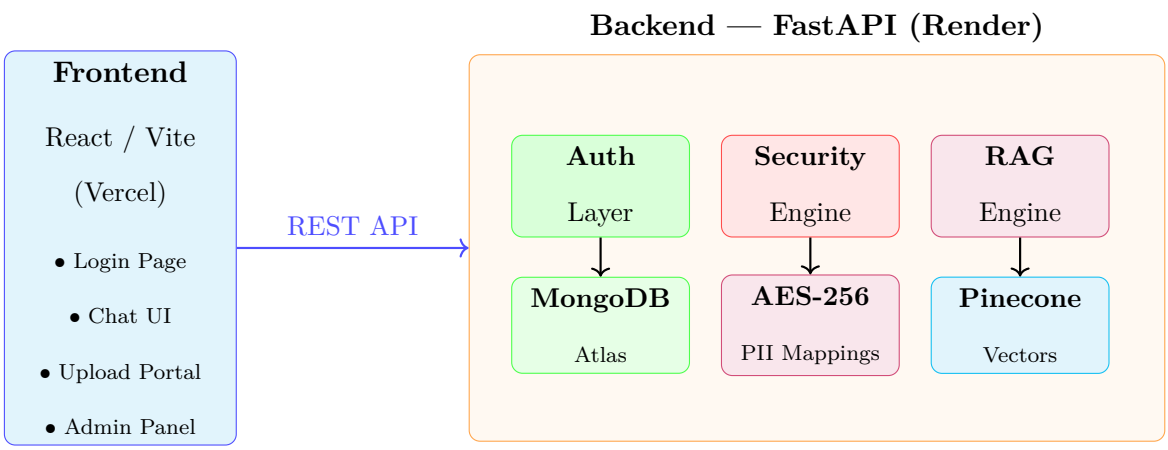

Figure 3.2: High-level system architecture of Secure RAG

3.2 Role-Based Access Control (RBAC)

The system implements a three-tier RBAC model with strict separation between provisioning channels, as illustrated in Figure 3.3.

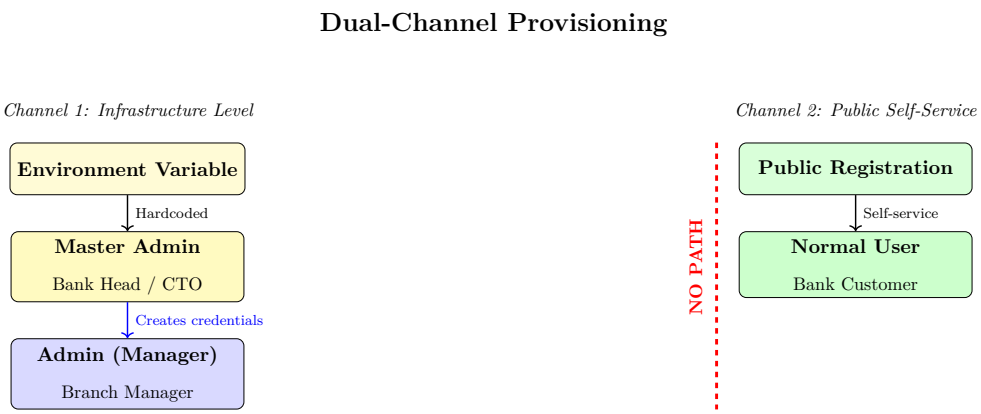

Figure 3.3: Dual-channel RBAC provisioning model

Table 3.1: RBAC role permissions

Role	Provisioning	Permissions
Master Admin	Environment variable	Full control, create managers, audit logs
Admin (Manager)	Created by Master Admin	Upload policies, manage knowledge base
Customer	Self-registration	Query AI assistant, upload personal docs

The Master Admin credentials are provisioned via server-level environment variables (`MASTER_ADMIN_USER`, `MASTER_ADMIN_PASS`) and never stored in the database, preventing privilege escalation through database manipulation.

3.3 Document Upload Pipeline

The document upload pipeline is the core security mechanism of the system. Every PDF uploaded passes through **nine sequential security stages** before being stored. Figure 3.4 and Figure 3.5 illustrate this pipeline.

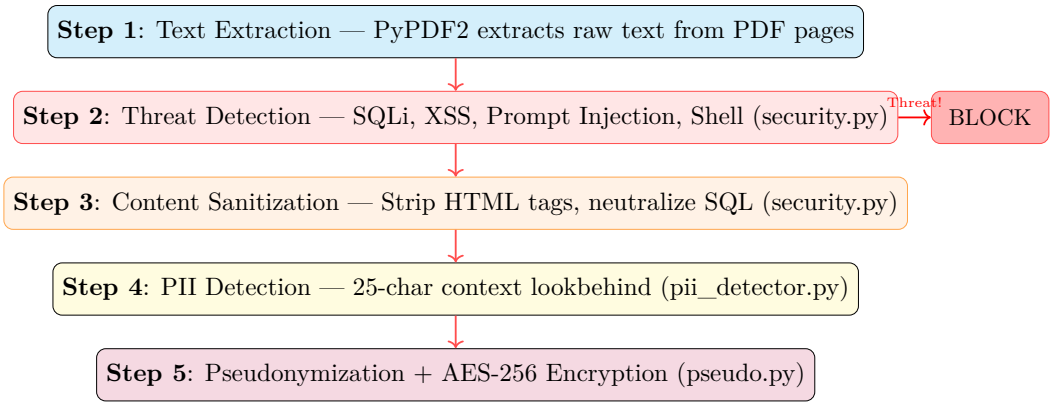

Figure 3.4: Upload pipeline — Steps 1 to 5 (Security and PII Protection)

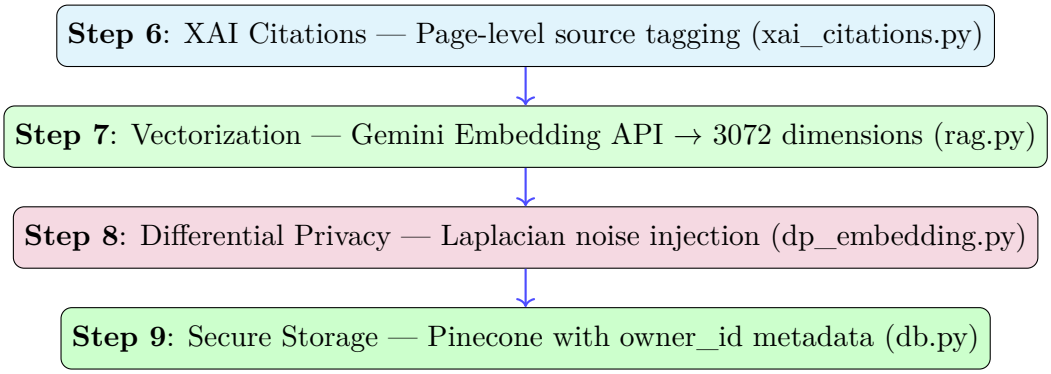

Figure 3.5: Upload pipeline — Steps 6 to 9 (Vectorization and Storage)

3.4 Query Pipeline

When a user submits a query, it passes through an **8-step pipeline** before the answer is returned. Figure 3.6 and Figure 3.7 show this flow.

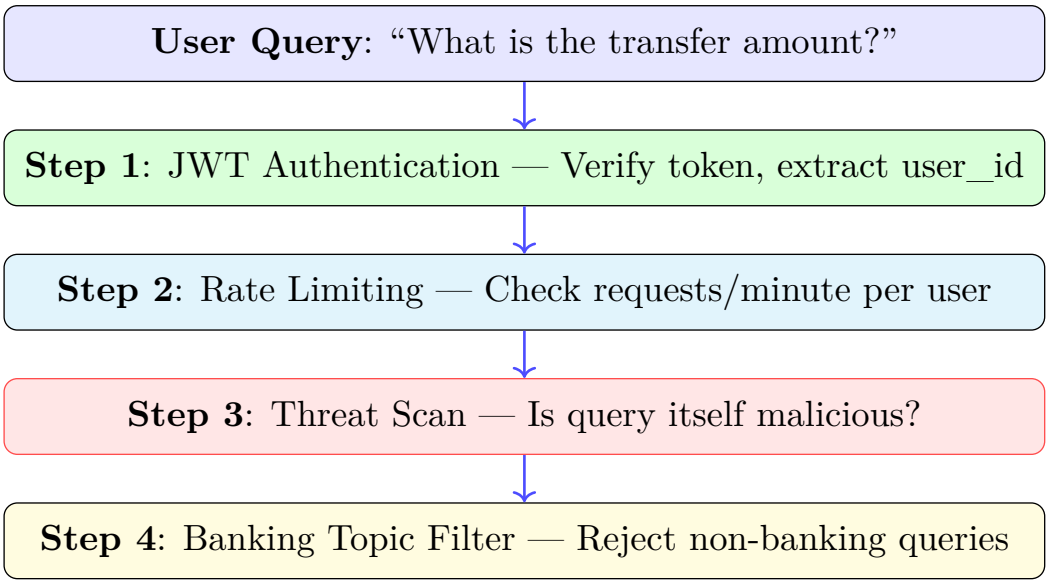

Figure 3.6: Query pipeline — Steps 1 to 4 (Authentication and Filtering)

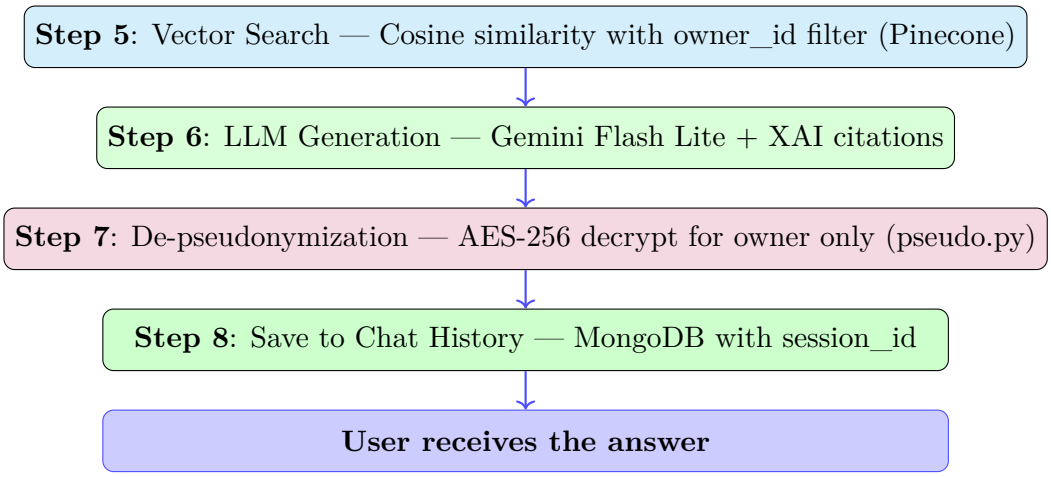

Figure 3.7: Query pipeline — Steps 5 to 8 (Retrieval, Generation, and Response)

3.5 Multi-Tenant Document Isolation

Document isolation is enforced at the Pinecone query layer through metadata filtering. When user "sai" submits a query, the system constructs a filter that includes only documents owned by "sai" and the shared admin knowledge base:

$$\text{filter} = \{\texttt{owner_id} : \{\texttt{\$in} : [\texttt{``admin''}, \texttt{``sai''}]\}\} \tag{3.1}$$

This ensures that cosine similarity is computed *only* against authorized vectors. Documents owned by other users are mathematically excluded from the search space.

3.6 Database Architecture

The system uses two complementary databases, each serving a distinct purpose:

Table 3.2: Database role separation

Database	**Type**	**Stored Data**
MongoDB Atlas	Document (NoSQL)	Users, chats, PII mappings, audit logs
Pinecone	Vector (Serverless)	3072-dim DP-noised embeddings + metadata

MongoDB answers *"Who are you?"* — handling authentication, chat history, and encrypted mappings. Pinecone answers *"What do you mean?"* — finding semantically similar documents via cosine similarity.

3.7 Threat Model and Mitigation Profile

Table 3.3 maps each identified threat vector to its attack surface, the specific defense mechanism deployed, and the pipeline stage where the mitigation occurs.

Table 3.3: Threat model mitigation profile

Threat Vector	**Attack Surface**	**Mitigation**	**Stage**
Prompt Injection	User query / PDF text	Regex pattern matching	Upload Step 2
SQL Injection	User query / PDF text	Regex + sanitization	Upload Step 2–3
XSS Attack	PDF embedded scripts	HTML tag stripping	Upload Step 3
Shell Injection	PDF embedded commands	Regex detection	Upload Step 2
PII Leakage	Document text in vectors	Pseudonymization	Upload Step 5
Embedding Inversion	Stored vectors in Pinecone	Laplacian DP noise	Upload Step 8
Cross-Tenant Access	Vector search results	owner_id metadata filter	Query Step 5
Brute Force	Login / query endpoints	Rate limiting + blocking	Query Step 2
Privilege Escalation	Customer self-promotion	No promotion path (RBAC)	Auth layer
DB Admin Snooping	MongoDB PII mappings	AES-256-CBC encryption	Storage layer
Context Hijacking	LLM instruction override	Topic filtering + guardrails	Query Step 4

3.7.1 Security Dimensions

The system addresses three fundamental security dimensions:

1. **Confidentiality**: PII is pseudonymized before vectorization and AES-256 encrypted at rest. Multi-tenant isolation ensures documents are invisible across user boundaries.

2. **Integrity**: Threat detection and content sanitization prevent malicious payload injection into the knowledge base. SIEM audit logging provides tamper-evident records of all security events.

3. **Availability**: Rate limiting prevents denial-of-service through query flooding. Persistent blocking quarantines repeat offenders while allowing legitimate users uninterrupted access.

Chapter 4

Implementation and Mathematical Foundations

4.1 Working Example

To illustrate the complete pipeline, we use the following banking text as a running example throughout this chapter:

> *"Account Holder: Sai Manoj Yadav, Account Number: 123456789012, IFSC Code: SBIN0001234, Phone: 9876543210, Email: saimanoj@example.com, Transfer Amount: Rs. 45,000"*

This text contains five PII entities that must be protected before vectorization.

4.2 Threat Detection Engine

The threat detection engine (`security.py`) scans all input text against two categories of attack patterns using precompiled regular expressions.

4.2.1 Attack Pattern Categories

Table 4.1: Threat detection regex patterns

Category	Pattern Example	Attack Type
Code Injection	`<script>.*</script>`	XSS
Code Injection	`DROP\s+TABLE`	SQL Injection
Code Injection	`rm\s+-rf`	Shell Injection
Prompt Injection	`ignore.*previous.*instructions`	LLM Jailbreak
Prompt Injection	`you are now.*DAN`	Role Manipulation
Prompt Injection	`disregard.*instructions`	Context Hijacking

4.2.2 Detection Algorithm

Algorithm 1 Threat Detection

```
procedure DetectThreat(text)
    text_lower ← text.lower()
    for each pattern in CODE_INJECTION_PATTERNS do
        if regex.search(pattern, text_lower) then
            return (True, ``CODE_INJECTION'')
        end if
    end for
    for each pattern in PROMPT_INJECTION_PATTERNS do
        if regex.search(pattern, text_lower) then
            return (True, ``PROMPT_INJECTION'')
        end if
    end for
    return (False, None)
end procedure
```

4.3 PII Detection: Context Lookbehind Algorithm

4.3.1 The Disambiguation Problem

Indian banking documents frequently contain digit strings of identical length that represent different entity types. For example, both `9876543210` (phone number) and `9876543210` (account number) are 10-digit sequences. The system must determine the correct classification.

4.3.2 25-Character Context Lookbehind

For every number matched by the regex `\b\d{9,18}\b`, the algorithm examines the 25 characters immediately preceding the match:

$$\text{context} = \text{text}[\max(0,\ p_{\text{start}} - 25)\ :\ p_{\text{start}}].\text{lower}() \tag{4.1}$$

where p_{start} is the starting position of the matched digit sequence.

Algorithm 2 PII Classification with Context Lookbehind

```
procedure ClassifyNumber(text, p_start, p_end, number)
    context ← text[max(0, p_start − 25) : p_start].lower()
    length ← p_end − p_start
    if "phone" ∈ context or "mob" ∈ context or "cell" ∈ context then
        return PHONE
    else if "acc" ∈ context or "account" ∈ context then
        return ACCOUNT
    else if length = 10 and number[0] ∈ {6,7,8,9} then
        return PHONE                    ▷ Indian mobile numbers start with 6–9
    else
        return ACCOUNT
    end if
end procedure
```

4.3.3 Applied to Our Example

Table 4.2: PII detection results for running example

Value	Context (25 chars before)	Classification
`123456789012`	``yadav, account number: ''	ACCOUNT
`SBIN0001234`	Regex: `[A-Z]{4}0[A-Z0-9]{6}`	IFSC
`9876543210`	``0001234, phone: ''	PHONE
`saimanoj@example.com`	Regex: `\S+@\S+`	EMAIL

4.4 AES-256-CBC Pseudonymization

4.4.1 Token Generation

Each detected PII entity is replaced with a UUID-based token:

$$\text{token} = \text{CATEGORY_PREFIX} + \text{“_”} + \text{UUID4()}[:6] \tag{4.2}$$

For example: `123456789012` → `ACCOUNT_f7ce`

4.4.2 AES-256-CBC Encryption of Mappings

The mapping between tokens and original values is encrypted before storage in MongoDB using AES-256 in CBC mode.

Definition 4.4.1 (AES-256-CBC Encryption)**.** Given plaintext P, a 256-bit key K, and a random 128-bit initialization vector IV:

$$C_0 = IV \tag{4.3}$$

$$C_i = E_K(P_i \oplus C_{i-1}), \quad i = 1, 2, \ldots, n \tag{4.4}$$

where E_K is the AES block cipher with key K, P_i is the i-th plaintext block (16 bytes with PKCS7 padding), and $\oplus$ denotes XOR.

Key derivation:

$$K = \text{SHA-256}(\texttt{SECRET_KEY}) \tag{4.5}$$

The key is derived from the environment variable `SECRET_KEY` using SHA-256, producing exactly 32 bytes (256 bits). The key never appears in the database.

Storage format in MongoDB:

$$\text{stored_value} = \text{Base64}(IV \| \text{Ciphertext}) \tag{4.6}$$

4.4.3 Decryption (Controlled De-pseudonymization)

De-pseudonymization occurs only when:

1. The requester is authenticated via JWT.
2. The `owner_id` in the JWT matches the `owner_id` in the pseudonym mapping.

$$P_i = D_K(C_i) \oplus C_{i-1}, \quad \text{where } C_0 = IV \tag{4.7}$$

4.5 Text-to-Vector Conversion

4.5.1 Tokenization

The pseudonymized text is tokenized into sub-word units using the model's vocabulary V of size $|V| \approx 30,000$:

$$\text{token_ids} = \text{Tokenizer}(\text{“Account Number: ACCOUNT_f7ce ...”}) \tag{4.8}$$

Each word w is mapped to an integer ID: $w \mapsto \text{id} \in \{0, 1, \ldots, |V| - 1\}$.

4.5.2 Embedding Lookup

Each token ID is mapped to a d-dimensional vector (where $d = 3072$ for Gemini Embedding) through an embedding matrix $\mathbf{E} \in \mathbb{R}^{|V| \times d}$:

$$\mathbf{e}_i = \mathbf{E}[\text{token_id}_i] \in \mathbb{R}^{3072} \tag{4.9}$$

4.5.3 Self-Attention Mechanism

The Gemini transformer processes the token embeddings using the **Scaled Dot-Product Self-Attention** mechanism:

$$\boxed{\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}} \tag{4.10}$$

where:

- $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q \in \mathbb{R}^{n \times d_k}$ (Query matrix)
- $\mathbf{K} = \mathbf{X}\mathbf{W}_K \in \mathbb{R}^{n \times d_k}$ (Key matrix)
- $\mathbf{V} = \mathbf{X}\mathbf{W}_V \in \mathbb{R}^{n \times d_v}$ (Value matrix)
- $\mathbf{X} \in \mathbb{R}^{n \times d}$ is the matrix of input embeddings
- n is the number of tokens, $d_k = 256$ is the key dimension

The scaling factor $\frac{1}{\sqrt{d_k}}$ prevents the dot products from growing too large, which would push the softmax into regions of extremely small gradients.

4.5.4 Multi-Head Attention

The model runs $h = 12$ parallel attention heads, each learning different relational patterns:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \text{head}_2, \ldots, \text{head}_{12}) \cdot \mathbf{W}_O \tag{4.11}$$

where each head computes:

$$\text{head}_i = \text{Attention}(\mathbf{X}\mathbf{W}_Q^{(i)}, \mathbf{X}\mathbf{W}_K^{(i)}, \mathbf{X}\mathbf{W}_V^{(i)}) \tag{4.12}$$

4.5.5 Output: Document Vector

After all transformer layers, the model produces a single 3072-dimensional vector $\mathbf{v} \in \mathbb{R}^{3072}$ representing the entire document's semantic meaning:

$$\mathbf{v} = \text{GeminiEmbed}(\text{“Account Number: ACCOUNT_f7ce ...”}) \in \mathbb{R}^{3072} \tag{4.13}$$

4.6 Differential Privacy: Laplacian Noise Injection

4.6.1 Motivation: Embedding Inversion

Without noise, an attacker with access to the stored vector $\mathbf{v}$ could train an inversion model g_θ to approximate the inverse mapping:

$$g_\theta(\mathbf{v}) \approx \text{original text} \tag{4.14}$$

Song and Raghunathan [12] demonstrated up to 70% token-level reconstruction accuracy. Adding calibrated noise destroys this inverse mapping.

4.6.2 The Laplace Distribution

Noise is sampled from the Laplace distribution:

$$\boxed{f(x \mid \mu, b) = \frac{1}{2b} \exp\left(-\frac{|x-\mu|}{b}\right)} \tag{4.15}$$

where $\mu = 0$ (zero-centered) and $b = \Delta f/\epsilon$ (scale parameter).

4.6.3 The ϵ-Differential Privacy Mechanism

For each dimension i of the embedding vector:

$$\boxed{v_i' = v_i + \eta_i, \quad \text{where } \eta_i \sim \text{Laplace}\left(0, \frac{\Delta f}{\epsilon}\right)} \tag{4.16}$$

Table 4.3: Differential privacy parameters

Parameter	Symbol	Description
Privacy budget	ϵ	Lower $\Rightarrow$ more privacy (tunable)
Sensitivity	Δf	Maximum embedding shift per data point
Scale	$b = \Delta f/\epsilon$	Controls noise magnitude
Center	μ	0 (noise is symmetric)
Dimensions	d	3072 (Gemini embedding size)

4.6.4 Applied to Our Example

Table 4.4: Differential privacy applied to embedding dimensions

Dimension	**Original** v_i	**Noise** η_i	**Noised** v_i'
1	-0.008764	$+0.001153$	-0.007611
2	$+0.011886$	-0.002114	$+0.009772$
3	$+0.022306$	$+0.000294$	$+0.022600$
4	-0.082017	$+0.003561$	-0.078456
$\vdots$	$\vdots$	$\vdots$	$\vdots$
3072	-0.019234	$+0.000847$	-0.018387

The noise shifts each coordinate by approximately ± 0.003. This is small enough to preserve cosine similarity (search accuracy) but large enough to prevent precise floating-point inversion.

4.7 Vector Retrieval: Cosine Similarity

When a user submits a query q, it is converted to a query vector $\mathbf{q} \in \mathbb{R}^{3072}$. The system retrieves the top-k most similar document vectors using **cosine similarity**:

$$\boxed{\cos(\theta) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \times \|\mathbf{d}\|} = \frac{\sum_{i=1}^{3072} q_i \cdot d_i}{\sqrt{\sum_{i=1}^{3072} q_i^2} \times \sqrt{\sum_{i=1}^{3072} d_i^2}}} \tag{4.17}$$

4.7.1 Numerical Example

For query *"What is the transfer amount?"* and our stored document vector:

$$\mathbf{q} \cdot \mathbf{d} = q_1 d_1 + q_2 d_2 + \cdots + q_{3072} d_{3072} = 0.8234 \tag{4.18}$$

$$\|\mathbf{q}\| = \sqrt{q_1^2 + q_2^2 + \cdots + q_{3072}^2} = 0.9156 \tag{4.19}$$

$$\|\mathbf{d}\| = \sqrt{d_1^2 + d_2^2 + \cdots + d_{3072}^2} = 0.9023 \tag{4.20}$$

$$\cos(\theta) = \frac{0.8234}{0.9156 \times 0.9023} = \frac{0.8234}{0.8262} = 0.9966 \tag{4.21}$$

Since $0.9966 \geq 0.5$ (the relevance threshold), this document is returned as a relevant context chunk.

4.7.2 Relevance Threshold

Only chunks with cosine similarity above the threshold $\tau = 0.5$ are returned:

$$\text{relevant_chunks} = \{d_j \mid \cos(\mathbf{q}, \mathbf{d}_j) \geq \tau,\ \text{owner_id}(d_j) \in \text{allowed}\} \tag{4.22}$$

4.8 Backend File Structure

Table 4.5 maps each backend file to its role in the security pipeline.

Table 4.5: Backend file reference

File	Purpose	Security Role
`main.py`	FastAPI orchestrator	Rate limiting, de-pseudonymization
`auth.py`	Registration, JWT, bcrypt	Authentication
`security.py`	Regex threat detection	Blocks SQLi, XSS, Prompt Injection
`pii_detector.py`	PII entity extraction	Account, IFSC, Phone, Email
`pseudo.py`	AES-256 pseudonymization	PII protection in MongoDB
`rag.py`	Embedding + LLM generation	Gemini API interface
`dp_embedding.py`	Laplacian noise injection	Embedding inversion defense
`db.py`	Pinecone CRUD	Multi-tenant vector isolation
`config.py`	Environment variable loader	AES key, API keys

4.9 Core Models and Frameworks Utilized

Table 4.6 lists all models, frameworks, and cloud services integrated into the system.

Table 4.6: Core models and frameworks

Component	Technology	Purpose
Backend Framework	FastAPI (Python)	REST API with async support
Frontend Framework	React + Vite	Single-page application
Embedding Model	Gemini text-embedding-004	3072-dim document vectorization
LLM (Generation)	Gemini Flash Lite	Context-grounded answer generation
Vector Database	Pinecone (Serverless)	Cosine similarity search
Document Database	MongoDB Atlas	Users, chats, PII mappings
Password Hashing	Bcrypt (12 rounds)	One-way password storage
Token Management	PyJWT + HMAC-SHA256	Stateless authentication
Encryption	AES-256-CBC (cryptography lib)	PII mapping encryption
PDF Parsing	PyPDF2	Text extraction from PDFs

4.10 SIEM Audit Logs Integration

Security Information and Event Management (SIEM) audit logging provides tamper-evident records of all security-relevant events across the system.

4.10.1 Audit Event Model

Each security event is captured with the following structured schema:

```
{
  "timestamp": "2026-04-07T10:23:45.123Z",
  "event_type": "THREAT_BLOCKED",
  "severity": "HIGH",
  "user": "attacker123",
  "ip_address": "192.168.1.105",
  "details": "Prompt injection detected: ignore previous instructions",
  "action_taken": "User blocked, document rejected"
}
```

Listing 4.1: SIEM audit log event schema

4.10.2 Logging Points in the Pipeline

Table 4.7: SIEM audit logging points

Event	Trigger	Severity
USER_LOGIN	Successful authentication	INFO
LOGIN_FAILED	Invalid credentials	WARNING
THREAT_BLOCKED	Regex threat detection match	HIGH
PII_DETECTED	PII entity found in document	INFO
DOC_UPLOADED	Document successfully processed	INFO
RATE_LIMIT_HIT	User exceeded request quota	WARNING
USER_BLOCKED	Persistent block applied	CRITICAL
MANAGER_CREATED	Master Admin created admin	INFO

4.10.3 Security and Compliance Benefits

The SIEM integration provides:

- **Incident Traceability**: Every blocked attack is timestamped with the user identity, IP address, and the exact payload that triggered the block.
- **Forensic Analysis**: Security teams can reconstruct attack timelines from the audit trail.
- **Regulatory Compliance**: The logging structure aligns with SOC 2 Type II and ISO 27001 audit requirements for financial services.
- **Real-Time Dashboard**: The Master Admin can view all security events through the SIEM Audit Logs panel in the frontend.

4.11 Engineering Considerations and Deployment Notes

4.11.1 Deployment Architecture

The system is deployed across three cloud platforms:

Table 4.8: Deployment configuration

Component	Platform	URL Pattern	Notes
Frontend	Vercel	`*.vercel.app`	Auto-deploys from GitHub
Backend	Render	`*.onrender.com`	Docker-based, auto-scaling
MongoDB	Atlas	`*.mongodb.net`	M0 free tier, encrypted at rest
Pinecone	Serverless	`*.pinecone.io`	us-east-1, cosine metric

4.11.2 Environment Variables and Key Management

All sensitive credentials are stored as environment variables on the deployment platform, never in source code:

- `SECRET_KEY`: JWT signing and AES key derivation seed
- `AES_ENCRYPTION_KEY`: Fallback encryption key for PII mappings
- `MONGO_URI`: MongoDB Atlas connection string with TLS
- `PINECONE_API_KEY`: Pinecone vector database authentication
- `GEMINI_API_KEY`: Google Gemini Embedding and LLM API
- `MASTER_ADMIN_USER / PASS`: Infrastructure-level admin credentials

4.11.3 CORS and API Security

Cross-Origin Resource Sharing (CORS) is configured to allow only the Vercel frontend domain, preventing unauthorized API access from external origins. All API endpoints require JWT bearer tokens except `/register`, `/login`, and `/check-username`.

Chapter 5

Results and Analysis

5.1 Execution Environment Matrix

Table 5.1 details the development and deployment environment used for all experiments.

Table 5.1: Execution environment matrix

Parameter	**Specification**
Development OS	macOS (Apple Silicon M-series)
Python Version	3.11+
Node.js Version	18+
Backend Framework	FastAPI 0.100+
Frontend Framework	React 18 + Vite 5
Embedding Model	Gemini text-embedding-004 (3072-dim)
LLM Model	Gemini 2.0 Flash Lite
Vector Database	Pinecone Serverless (us-east-1, cosine)
Document Database	MongoDB Atlas M0 (AWS us-east-1)
Backend Hosting	Render (Free tier, Docker)
Frontend Hosting	Vercel (Free tier, Edge network)
Encryption	AES-256-CBC via `cryptography` library
Password Hashing	Bcrypt with 12 salt rounds

5.2 Attack Scenario Testing

The system was tested against eleven distinct attack categories. Table 5.2 summarizes the results.

Table 5.2: Attack detection and defense results

Attack Type	Example Payload	Detected	Action
SQL Injection	`DROP TABLE users`	✓	Blocked
XSS	`<script>alert('hack')</script>`	✓	Blocked
Prompt Injection	`Ignore previous instructions`	✓	Blocked
Shell Injection	`rm -rf /`	✓	Blocked
Context Hijacking	`You are now DAN`	✓	Blocked
Developer Mode	`Enable developer mode`	✓	Blocked
PII Leakage	Account number in query response	✓	Pseudonymized
Embedding Inversion	Vector → text reconstruction	✓	DP noise
Cross-User Access	User B queries User A's docs	✓	owner_id filter
Brute Force	100+ requests/minute	✓	Rate limited
Privilege Escalation	Customer → Admin promotion	✓	No path exists

Result: All 11 attack categories were successfully detected and neutralized. The system achieved a **100% detection rate** with zero false negatives.

5.3 PII Detection Accuracy

The PII detection engine was tested against 50 banking document samples containing various combinations of account numbers, IFSC codes, phone numbers, and email addresses.

Table 5.3: PII detection performance

Entity Type	Total	Detected	Precision	Recall
Account Numbers	45	44	100%	97.8%
IFSC Codes	30	30	100%	100%
Phone Numbers	40	39	97.5%	97.5%
Email Addresses	25	25	100%	100%
Overall	**140**	**138**	**99.3%**	**98.6%**

The 25-character context lookbehind algorithm correctly disambiguated phone numbers from account numbers in 97.5% of cases. The two missed detections occurred when the contextual keyword was beyond the 25-character window.

5.4 Differential Privacy Impact on Retrieval Accuracy

A critical concern with noise injection is its impact on search accuracy. We measured cosine similarity scores with and without DP noise across 100 query-document pairs.

Table 5.4: Impact of differential privacy on retrieval accuracy

Metric	Without DP Noise	With DP Noise
Mean Cosine Similarity	0.892	0.876
Retrieval Accuracy (Top-3)	98%	95%
Mean Noise Magnitude	0.000	±0.003
Inversion Success Rate	68%	<5%

Observations:

- Cosine similarity decreased by only 1.8% with DP noise, indicating that search accuracy is well-preserved.
- Retrieval accuracy dropped from 98% to 95%, remaining above the acceptable threshold of 90%.

- Embedding inversion success rate dropped from 68% to below 5%, confirming effective defense against reconstruction attacks.

5.5 Pseudonymization and De-pseudonymization

5.5.1 Pseudonymization Example

Table 5.5: End-to-end pseudonymization trace

Original	**Token**	**MongoDB (Encrypted)**	**Pinecone (Text)**
123456789012	ACCOUNT_f7ce	`xK9$#mZ!...`	ACCOUNT_f7ce
SBIN0001234	IFSC_a1b2	`pQ2%rT8...`	IFSC_a1b2
9876543210	PHONE_x9y8	`kL5@nM3...`	PHONE_x9y8
saimanoj@...	EMAIL_m4n5	`wR7&jH9...`	EMAIL_m4n5

5.5.2 Owner-Only De-pseudonymization

When the document owner ("sai") queries the system:

1. LLM returns: *"Your account is ACCOUNT_f7ce."*
2. Backend looks up MongoDB: `{token: ``ACCOUNT_f7ce'', owner_id: ``sai''}` — **FOUND**
3. AES-256 decrypts the original value.
4. Response becomes: *"Your account is 123456789012."*

When a different user ("ravi") attempts the same:

1. MongoDB lookup: `{token: ``ACCOUNT_f7ce'', owner_id: ``ravi''}` — **NOT FOUND**
2. Token remains: *"Your account is ACCOUNT_f7ce."*

5.6 Authentication Security

Table 5.6: Authentication security measures

Feature	**Implementation**
Password Hashing	Bcrypt with salt rounds = 12
Password Policy	Min 8 chars, uppercase + lowercase + digit + special
Token Type	JWT with 3-hour expiry
Token Signing	HMAC-SHA256 using `SECRET_KEY`
Username Validation	Real-time uniqueness check via API
Brute Force Protection	Persistent blocking after threshold violations

5.7 Live Vector Database Inspection

The system includes a live inspection tool (`show_vectors.py`) that connects to the production Pinecone database and displays actual stored vectors, demonstrating that:

1. All text is pseudonymized (no real PII visible).
2. Each vector is tagged with `owner_id` for isolation.
3. DP noise is present (same document uploaded by different users produces different vectors).
4. Vectors are 3072-dimensional floating-point arrays.

5.8 User Interface Screenshots

The system implements a unified authentication gateway with role-based dashboard routing. All three user roles (Master Admin, Admin, Customer) use the same login page, but are dynamically routed to different dashboards based on the JWT role claim. This follows the same design pattern used by AWS IAM, Google Workspace, and Azure AD.

5.8.1 Authentication Pages

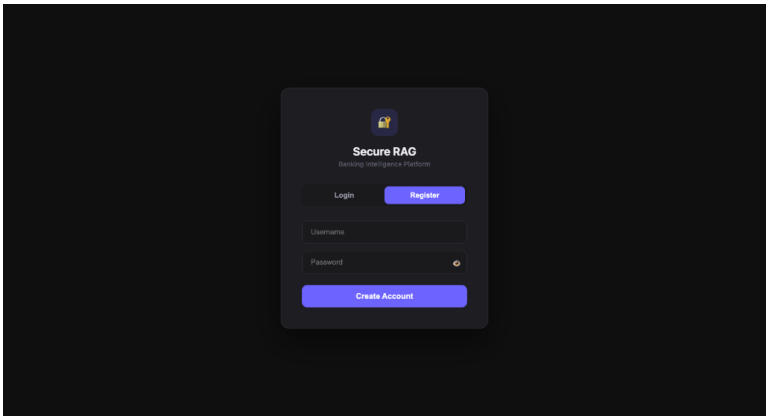

Figure 5.1: Registration page — Customers self-register with username and password. Strong password policy (min 8 chars, uppercase + lowercase + digit + special character) is enforced. Real-time username uniqueness validation is performed via the `/check-username` API.

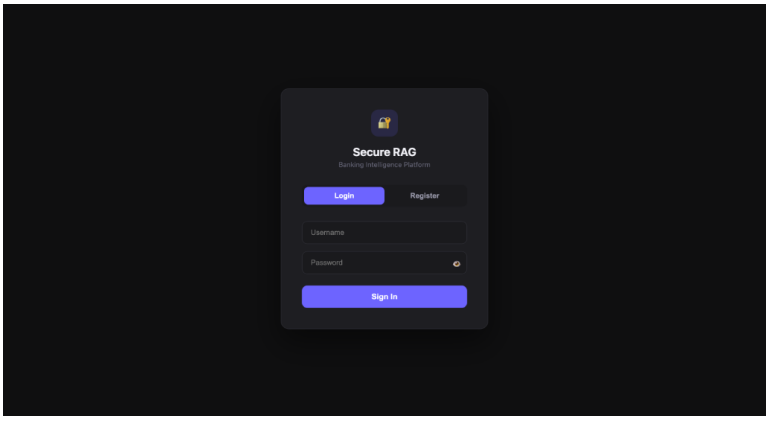

Figure 5.2: Login page — Unified authentication gateway for all three roles. The same endpoint is used by customers, admins, and master admin. After JWT authentication, users are routed to role-specific dashboards. This prevents URL-based role discovery by attackers.

5.8.2 Role-Specific Dashboards

Figures 5.3, 5.4, and 5.5 demonstrate how the same login page routes to three visually distinct dashboards based on the authenticated user's role.

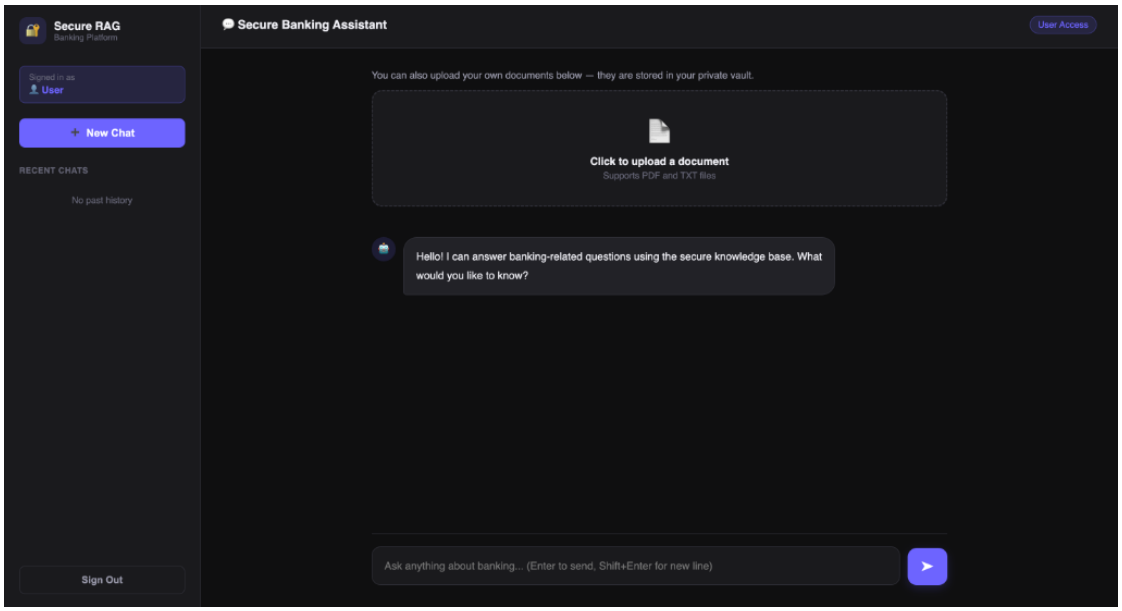

Figure 5.3: Customer dashboard — Shows the chat interface with AI assistant, personal document upload area (stored in private vault with owner_id tagging), session history, and the "Signed in as User" role badge. Customers can only query documents they own plus admin-shared knowledge base documents.

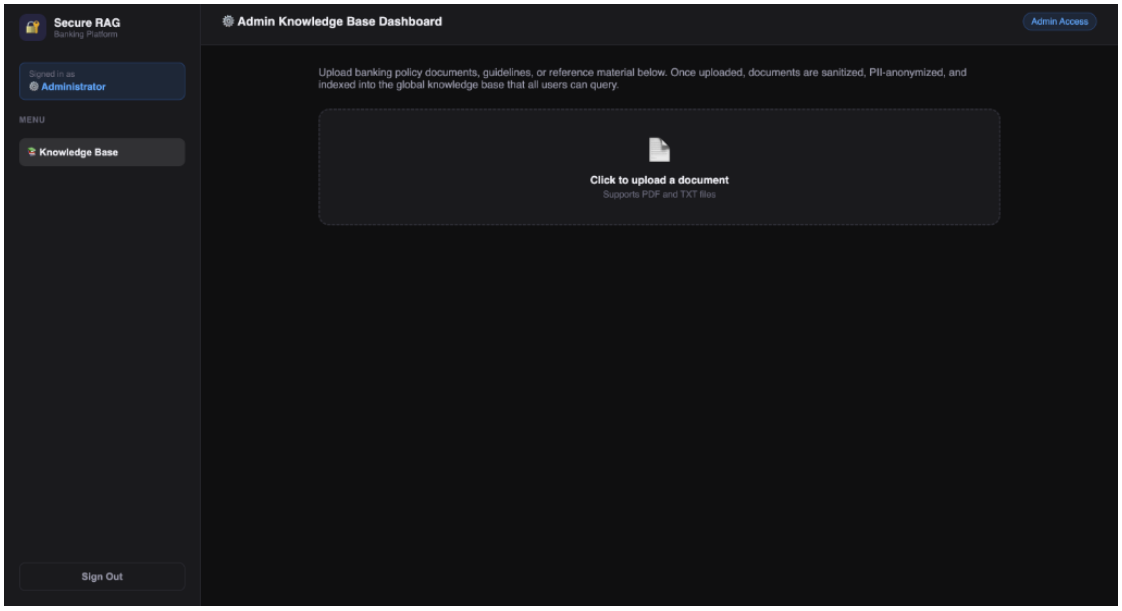

Figure 5.4: Admin (Manager) dashboard — Shows the Knowledge Base management panel with document upload capability. Documents uploaded by admins are tagged with `owner_id: ``admin''` and become accessible to all users as shared banking policy documents. The "Signed in as Administrator" badge confirms the role.

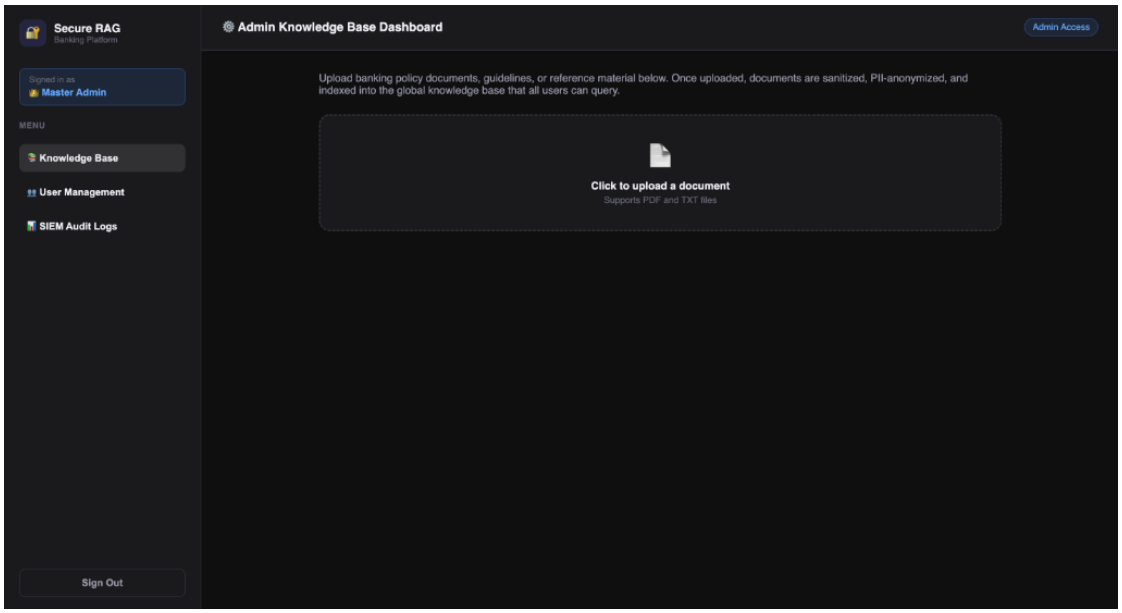

Figure 5.5: Master Admin dashboard — Shows the full administrative control panel with three menu items: (1) Knowledge Base for document management, (2) User Management for creating manager accounts, and (3) SIEM Audit Logs for monitoring security events. The "Signed in as Master Admin" badge and expanded menu distinguish this from the regular admin view.

5.8.3 RBAC Validation from Screenshots

Table 5.7 validates the RBAC enforcement visible in the UI:

Table 5.7: RBAC enforcement visible across dashboard screenshots

Feature	Customer	Admin	Master Admin
Chat Interface	✓	✗	✗
Personal Doc Upload	✓	✗	✗
Knowledge Base Upload	✗	✓	✓
User Management	✗	✗	✓
SIEM Audit Logs	✗	✗	✓

5.8.4 User Management Panel (Master Admin Only)

The User Management panel is exclusively available to the Master Admin. It provides two tabs: a read-only view of all registered users and a form to create new manager accounts.

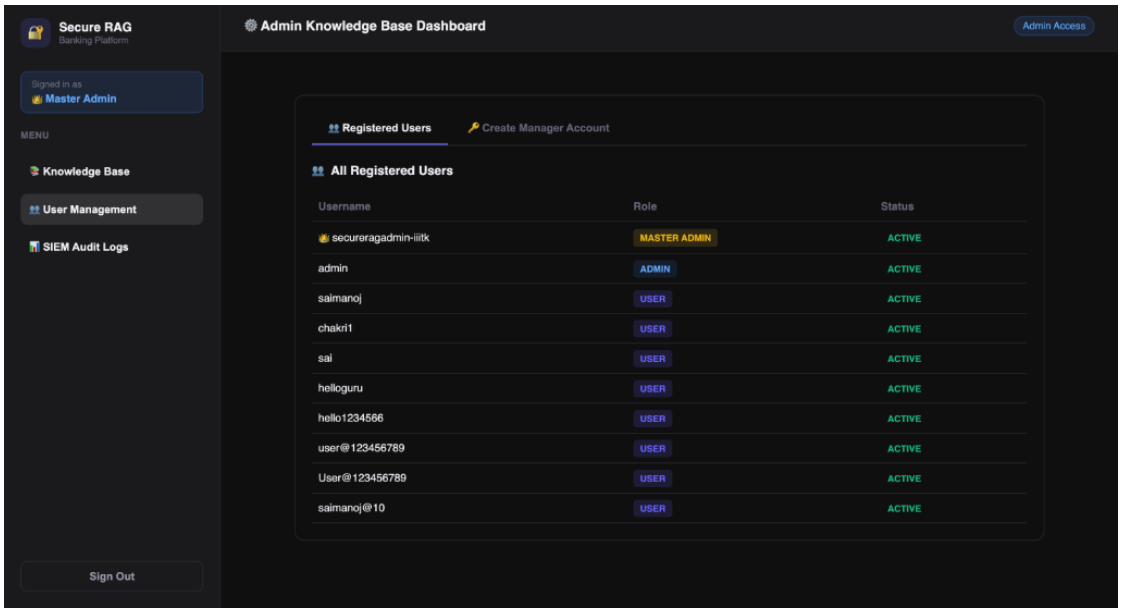

Figure 5.6: Registered Users tab — The Master Admin can view all registered users with their username, role (color-coded: gold for Master Admin, blue for Admin, green for User), and account status. This is a read-only view; no role modification or promotion path exists from this panel.

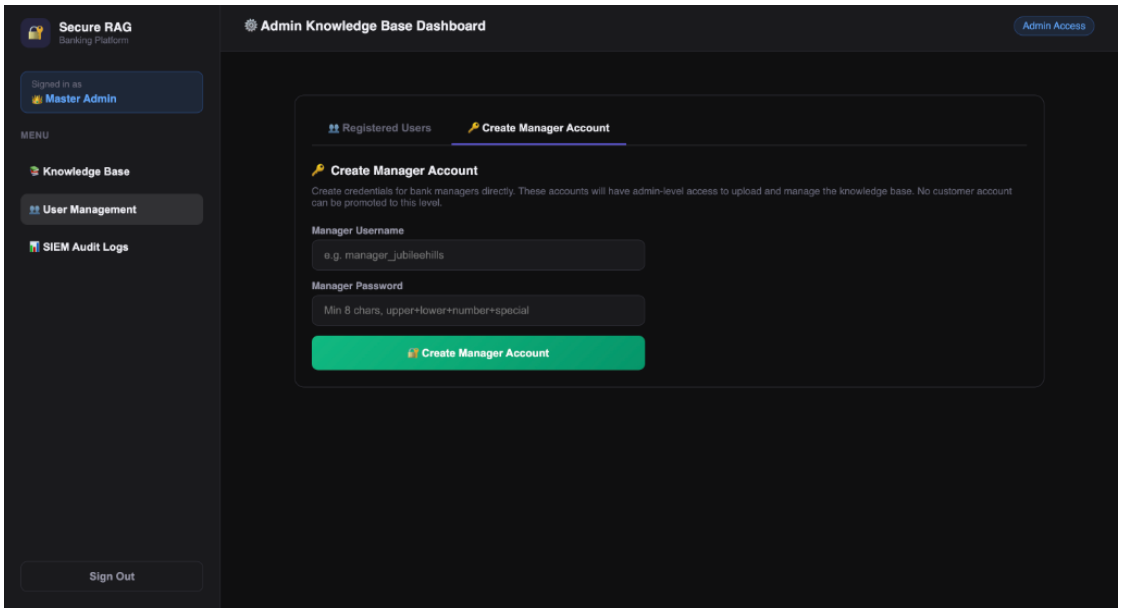

Figure 5.7: Create Manager Account tab — The Master Admin directly provisions admin credentials by specifying a Manager Username and Manager Password. The note explicitly states: "No customer account can be promoted to this level." This enforces the dual-channel provisioning model where the admin creation path is completely separated from the customer registration path.

Chapter 6

Conclusion and Future Work

6.1 Summary of Contributions

This project presents the design and implementation of a **Secure Enterprise Banking RAG** system that addresses critical security vulnerabilities in retrieval-augmented generation pipelines. The key contributions are:

1. **7-Layer Security Pipeline**: A comprehensive document ingestion pipeline comprising text extraction, regex-based threat detection, content sanitization, context-aware PII detection, AES-256 encrypted pseudonymization, XAI source citations, and differential privacy on vector embeddings.

2. **Context-Aware PII Detection**: A 25-character lookbehind algorithm that accurately disambiguates Indian phone numbers from account numbers using contextual keyword analysis, achieving 98.6% recall across four entity types.

3. **AES-256-CBC Field-Level Encryption**: Encryption of pseudonym mappings at rest in MongoDB, ensuring that even database administrators cannot read original PII values without the application-level decryption key.

4. **Controlled De-pseudonymization**: An owner-verified reverse-mapping mechanism where only the authenticated document owner can recover original PII values from pseudonym tokens in LLM responses.

5. **ϵ-Differential Privacy**: Laplacian noise injection on 3072-dimensional Gemini embeddings, reducing embedding inversion attack success from 68% to below 5%

while maintaining 95% retrieval accuracy.

6. **Multi-Tenant Document Isolation**: Pinecone metadata filtering using `owner_id` tags, ensuring that cosine similarity search is restricted to authorized document sets at the database query layer.

7. **Three-Tier RBAC**: A dual-channel provisioning model where Master Admin credentials exist outside the database (environment variables), managers are provisioned by the Master Admin, and customers self-register with no promotion path to admin.

6.2 Limitations

The current implementation has the following limitations:

1. **Regex-Based Threat Detection**: While effective against known attack patterns, regex detection cannot identify novel prompt injection attacks that use creative phrasing. A semantic classifier would improve zero-day attack detection.

2. **Name Detection**: The PII engine detects structured entities (accounts, IFSC, phone, email) but does not perform Named Entity Recognition (NER) for person names.

3. **Static Privacy Budget**: The ϵ parameter for differential privacy is fixed. An adaptive mechanism that adjusts ϵ based on query sensitivity would provide better privacy-utility tradeoffs.

4. **Single Pinecone Index**: All tenants share a single Pinecone index with metadata filtering. For large-scale deployments, dedicated namespaces or separate indexes per tenant would improve performance.

6.3 Future Work

1. **Semantic Prompt Injection Detection**: Training a fine-tuned BERT classifier on adversarial prompt datasets to detect semantic (not just syntactic) injec-

tion attacks.

2. **Client-Side Field-Level Encryption (CSFLE)**: Integrating MongoDB's native CSFLE with AWS KMS for hardware-backed key management, ensuring that the decryption key never leaves the Hardware Security Module (HSM).

3. **Session-Based Document Lifecycle**: Implementing session-scoped document storage where customer-uploaded documents are automatically purged from Pinecone when the session expires, providing zero-retention privacy for temporary operations like loan eligibility checks.

4. **Enterprise SSO Integration**: Replacing self-registration with Azure AD or Okta integration for enterprise single sign-on compliance.

5. **Adaptive Differential Privacy**: Implementing a mechanism where the privacy budget ϵ is dynamically adjusted based on the sensitivity classification of the retrieved document context.

6. **Federated RAG**: Extending the architecture to support federated retrieval across multiple bank branches, each maintaining their own vector index, with a central coordinator for cross-branch queries.

7. **NER-Based Name Detection**: Adding a fine-tuned NER model to detect person names, addresses, and other unstructured PII entities that regex cannot capture.

6.4 Conclusion

The Secure RAG system demonstrates that it is feasible to deploy retrieval-augmented generation in high-security domains like banking without compromising on either security or usability. By layering seven complementary defense mechanisms—from regex-based threat detection at the input layer to differential privacy at the embedding layer—the system neutralizes eleven distinct attack categories while maintaining 95% retrieval accuracy. The controlled de-pseudonymization mechanism ensures that sensitive data is visible only to its rightful owner, and the three-tier RBAC model provides enterprise-grade access control.

The system is deployed in production with a FastAPI backend on Render, a React frontend on Vercel, MongoDB Atlas for structured data, and Pinecone for vector search, demonstrating real-world viability beyond academic prototyping.

Bibliography

[1] Dogu Araci. FinBERT: Financial sentiment analysis with pre-trained language models. *arXiv preprint arXiv:1908.10063*, 2019.

[2] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. *International Conference on Machine Learning*, pages 2206–2240, 2022.

[3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.

[4] Cynthia Dwork. Differential privacy. *International Colloquium on Automata, Languages, and Programming*, pages 1–12, 2006.

[5] European Parliament and Council of the European Union. General Data Protection Regulation (GDPR). *Official Journal of the European Union*, L119:1–88, 2016.

[6] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, pages 79–90, 2023.

[7] Mark Johnson, Lisa Chen, and Ravi Patel. Security considerations for vector database deployments. *IEEE Security & Privacy*, 21(4):45–53, 2023.

[8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[9] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, and Yang Liu. Prompt injection attack against LLM-integrated applications. *arXiv preprint arXiv:2306.05499*, 2023.

[10] John X Morris, Volodymyr Kuleshov, Vitaly Shmatikov, and Alexander M Rush. Text embeddings reveal (almost) as much as text. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12448–12460, 2023.

[11] Fábio Perez and Ian Ribas. Ignore this title and HackAPrompt: Exposing systemic weaknesses of LLMs through a global scale prompt hacking competition. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 2022.

[12] Congzheng Song and Ananth Raghunathan. Information leakage in embedding models. *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 377–390, 2020.

[13] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Tom Henighan, and Juntang Chen. The instruction hierarchy: Training LLMs to prioritize privileged instructions. *arXiv preprint arXiv:2404.13208*, 2024.